

Reinforcement Learning Framework Deployment

Step-by-Step Guide for Godot 4 (.NET Edition) & Python Toolchains on macOS

1. System Architecture Overview

The `godot_rl_agents` ecosystem functions as a split architecture. The setup coordinates two fully distinct runtimes interacting over a local loopback network socket connection:

Component	Responsibilities	Runtime / Language
Environment (Godot)	Renders nodes, processes physics, collects observations, and enforces rewards.	Godot .NET Editor Engine / Custom code written in GDScript .
Core Framework Backend	Compiles native C# tasks and bridges Godot to specialized model parsers.	C# Integration Layer via Microsoft .NET CoreCLR .
Brain Runtime (Python)	Executes optimization algorithms (PPO, cleanRL) and fits internal tensor policies.	Isolated Virtual Environment via Conda (Python 3.10) .

Crucial Requirement: You must execute this workflow utilizing the **Godot Engine .NET Edition**. The standard edition lacks the capability to parse the external dependencies and NuGet references compiled into the plugin layer, which handles local tensor execution.

2. Conda Environment Provisioning

To preserve local runtime purity and guarantee consistent package interactions, isolate your machine learning packages inside a dedicated Conda virtual workspace running Python 3.10.

Step 2.1: Create the Isolated Environment

Open your macOS Terminal application and build the environment shell:

```
conda create --name godot_env python=3.10 -y
```

Step 2.2: Activate the Local Scope

Target your terminal instance explicitly to point to the freshly generated space:

```
conda activate godot_env
```

Step 2.3: Install Framework Packages

Install the base framework along with Stable Baselines3 wrappers. Double quotes ensure matching shells do not misinterpret system syntax flags:

```
pip install "godot-rl[sb3]"
```

3. Godot Project Configuration

Prepare your local project structure to reference and resolve the internal C# hooks required by the native plugin structure.

Step 3.1: Open and Extract the Target Example

Extract the `godot_rl_JumperHard` source tree. Launch your **Godot .NET Editor**, click **Import**, locate the directory, and select its matching `project.godot` descriptor file.

Step 3.2: Enable the Plugin Hooks

Navigate within top menus to **Project** → **Project Settings** → **Plugins**. Locate the entry for *Godot RL Agents* and toggle the status to **Enabled**. Let the background MSBuild process complete its structural system evaluation.

4. Interactive In-Editor Training Workflow

Because pre-compiled platform binaries are heavily architecture-dependent, the recommended approach on macOS machines is **In-Editor Live Interactive Training** via socket bindings.

Step 4.1: Launch the Python Training Listener

In your activated terminal, instruct the server to run. Appending the optimization flags prepares the model checkpoints to save out when milestones are hit:

```
gdrl --experiment_name=Jumper_Mac --viz --save_model_path=jumper_brain --  
onnx_export_path=jumper_brain.onnx
```

The console window will freeze and enter a holding pattern, reading: *"Waiting for connection from Godot..."*

Step 4.2: Signal the Handshake From Godot

Return to the open Godot .NET Editor window. Double-click the primary training layout scene (e.g., `main.tscn` or `train.tscn`) in the FileSystem tree. Press **Play Scene (F6)**.

Behavioral Psychology & Reward Shaping Note: The agent optimizes its weights using iterative reinforcement signals. The reward function calculates $R = r_{\{target\}} + r_{\{step\}} - r_{\{penalty\}}$. To shape faster behavioral adaptation, select the individual Agent nodes and verify how reward intervals correlate with structural milestones.

5. Exporting and Deploying the Trained Model

Once training loops display sufficient structural converging behavior, you can decouple your runtime entirely from Python and execute your code in production.

Step 5.1: Extract the Artifact

Upon completing or interrupting your terminal session via `Ctrl + C`, find the generated `jumper_brain.onnx` file within your active terminal execution path.

Step 5.2: File Injection

Drag the `jumper_brain.onnx` file straight into Godot's FileSystem pane, placing it cleanly into your project's resource root (`res://`).

Step 5.3: Structural Node Re-targeting

Select the main **Sync** node in your active Godot Scene view hierarchy. Inside the Inspector panel on the right side of the editor, adjust the properties as follows:

- Change the **Control Mode** option dropdown selection from *Training* to **ONNX Inference**.
- Assign your newly generated `res://jumper_brain.onnx` to the empty **ONNX Model Path** property field slot.

Test the implementation by running your scene standalone. Your game agents will instantly carry out their behaviors locally using the compiled C# OnnxRuntime layer, functioning with zero dependency on active Python tasks or background Conda terminal environments.